NAT 단일 장애점 제거와 통합 모니터링 체계 구축

접수 마감 트래픽 대응 인프라의 가용성 개선 제안

제안사 | Beaver Works 2조
고객사 | Ferret 6조

해커톤: 2팀 Beaver
김영배 박진호 양희연 오영빈 정재우 정혜빈

목차

① 고객 분석 및 요구사항

1.1 고객사 선정 배경 03

1.2 수행 방식 및 일정 04

1.3 현황 분석 05

1.4 요구사항 정의 08

② 솔루션 설계

2.1 요구사항별 솔루션 09

2.2 As-Is · To-Be 10

2.3 기술 선택 근거 13

③ 구현 및 검증

3.1 구현 결과 요약 14

3.2 시연 환경 세팅 18

3.3 시연 영상 20

④ 도입 효과 및 운영

4.1 개선 효과 21

4.2 비용 및 운영 방안 22

1.1 고객사 선정 배경 6조가 꼽은 개선 과제 중, 우리가 구축하고 검증한 범위에서 제안

우리는

BEAVER WORKS

고객사

6조

Ferret 팀

[클라우드 인프라 자동화 및 최적화 구축]
접수 마감 트래픽 폭주 대응을 위한 하이브리드 기반 프로젝트

고객사 선정의 근거가 되는 경험

① Multi-AZ NAT 이중화 구성

비용 절감 목적으로 NAT Gateway 대신 **NAT Instance를 2a/2c에 1대씩 배치**
워커 노드, RDS도 AZ 분산

② 장애 주입 시나리오 테스트

파드, 노드, AZ, DB 장애 주입 후 복구 시간과 데이터 유실 측정
AZ 장애 50초 내 복구, 주문 유실 0건

고객사 선정 이유

① 고객이 꼽은 과제가 우리가 구축하고 검증해 본 영역

추후 개선점 1번(단일 장애점 제거), 5번(모니터링 통합)

아쉬운 점	추후 개선점
1. 초반 설계가 미흡해 수정 사항이 많았다. 2.업무 분배에 아쉬움이 있다. 3.제한된 온프렘 환경에서는 실제 트래픽 분산을 충분히 재현하지 못했다. 4.보안 권한 설정과 Terraform 코드 배포, 대시보드 구성에서 처음부터 체계적으로 갖추지 못한 부분이 남았다.	1. 단일 장애점 제거 2.데이터 보호 강화 3.코드 기준 운영 완성 4.보안 최소 권한 정리 5.모니터링 통합 6.자원 제약에 따른 테스트 설계

② 비슷한 비용 조건에서 설계해 본 경험

우리도 비용 때문에 NAT Gateway 대신 NAT Instance로 이중화
→ 저예산으로 구성한 고객사의 환경에도 적용 가능

1.2 수행 방식 및 일정

11/02 Day 01
- 6조 결과보고서 분석
- 담당자 면담, 요구사항 확정
- 아키텍처 설계

11/03 Day 02
- 6조 AWS 네트워크 구조 재현
- R1, R2 구현
- 부하 테스트 진행

11/04 Day 03
- 검증 결과 정리
- 발표 자료와 시연 영상 제작

6조 미팅 관련 자료 공유

2팀

6조의 기본 프로젝트 산출물과 회고를 바탕으로 개선 방안을 작성했습니다.

저희 팀은 기본 프로젝트에서 DB 장애 전환과 데이터 정합성 검증(유실 0건, 중복 0건)을 수행했고,
인프라를 코드로 구성하고 운영한 경험이 있습니다. 이 경험을 살려 **DB 이중화와 백업 복구 검증 자동화**를 목표로
개선 방안을 생각 중입니다.

ID	요구사항	6조 회고)	우선순위
R-01	DB 장애가 나도 서비스와 데이터가 유지되어야 한다	추후 개선점 1	Must
R-02	접수 데이터 손실 범위를 줄이고, 백업이 실제로 복구되는지 보장해야 한다	추후 개선점 2	Must
R-03	콘솔 수작업 없이 코드로 배포하고 운영해야 한다	추후 개선점 3 / 아쉬운 점 4	Must

질문 사항

1. **저희가 생각한 방향과 별개로, 생각하시는 개선 방향이나 요구사항이 있으신가요? 있다면 우선순위가 어떻게 되나요?**
2. (허용 가능한 RPO와 RTO는 어느 정도인가요?)
3. (월 예산 상한은 얼마인가요? / ppt 내용은 확인했는데 혹시 상한을 염두에 두고 계시는지 궁금합니다.)

수행 방식

산출물 분석 → 담당자 미팅 → 환경 재현 및 구현 → 개선 효과 검증

① 고객사와 미팅

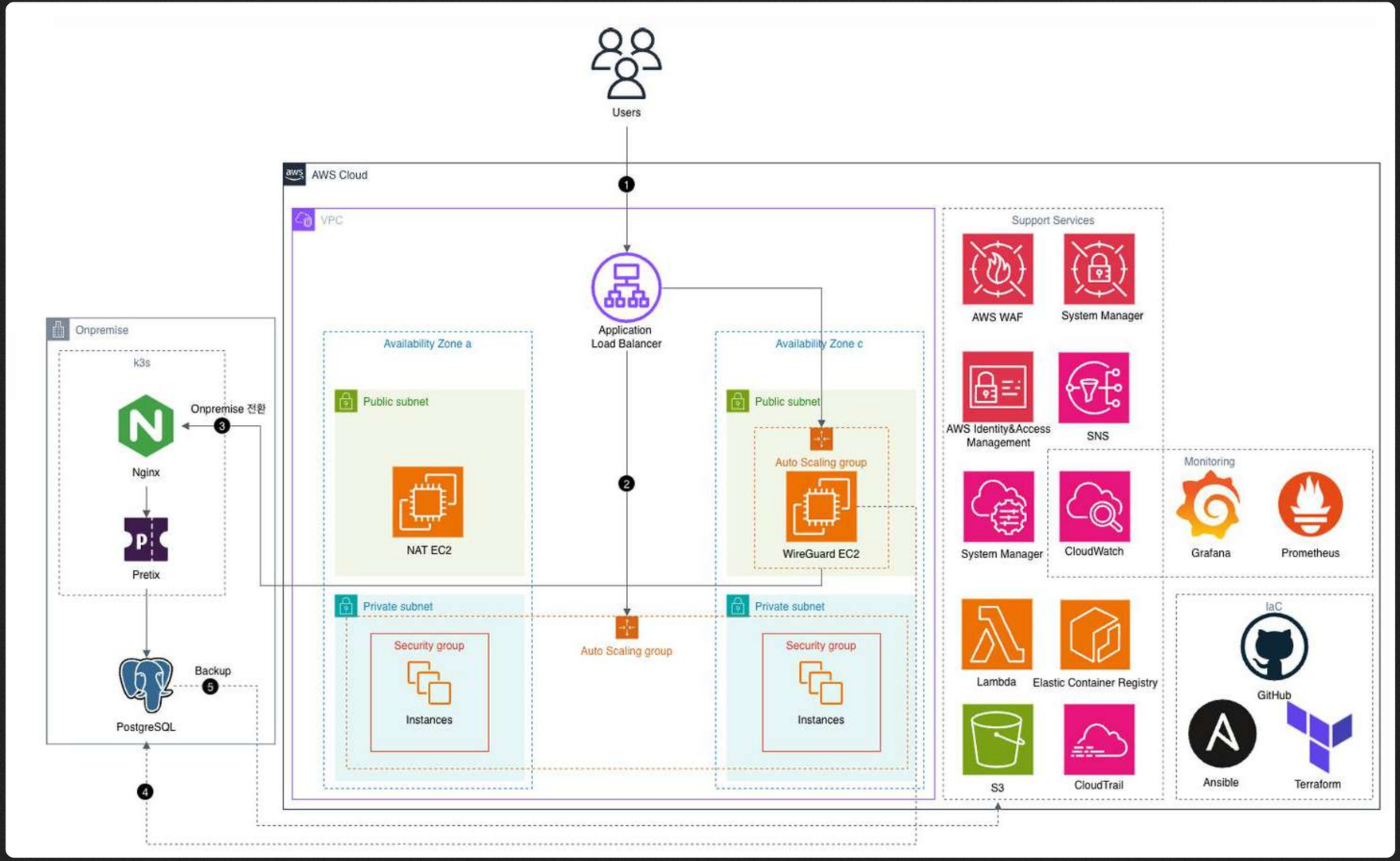
고객사 발표 자료를 바탕으로 개선안 초안 작성 후 담당자 면담

→ 초안의 DB 이중화 방향은 미팅 결과를 반영해 제외하고, NAT 이중화와 모니터링 통합으로 프로젝트 범위를 확정함

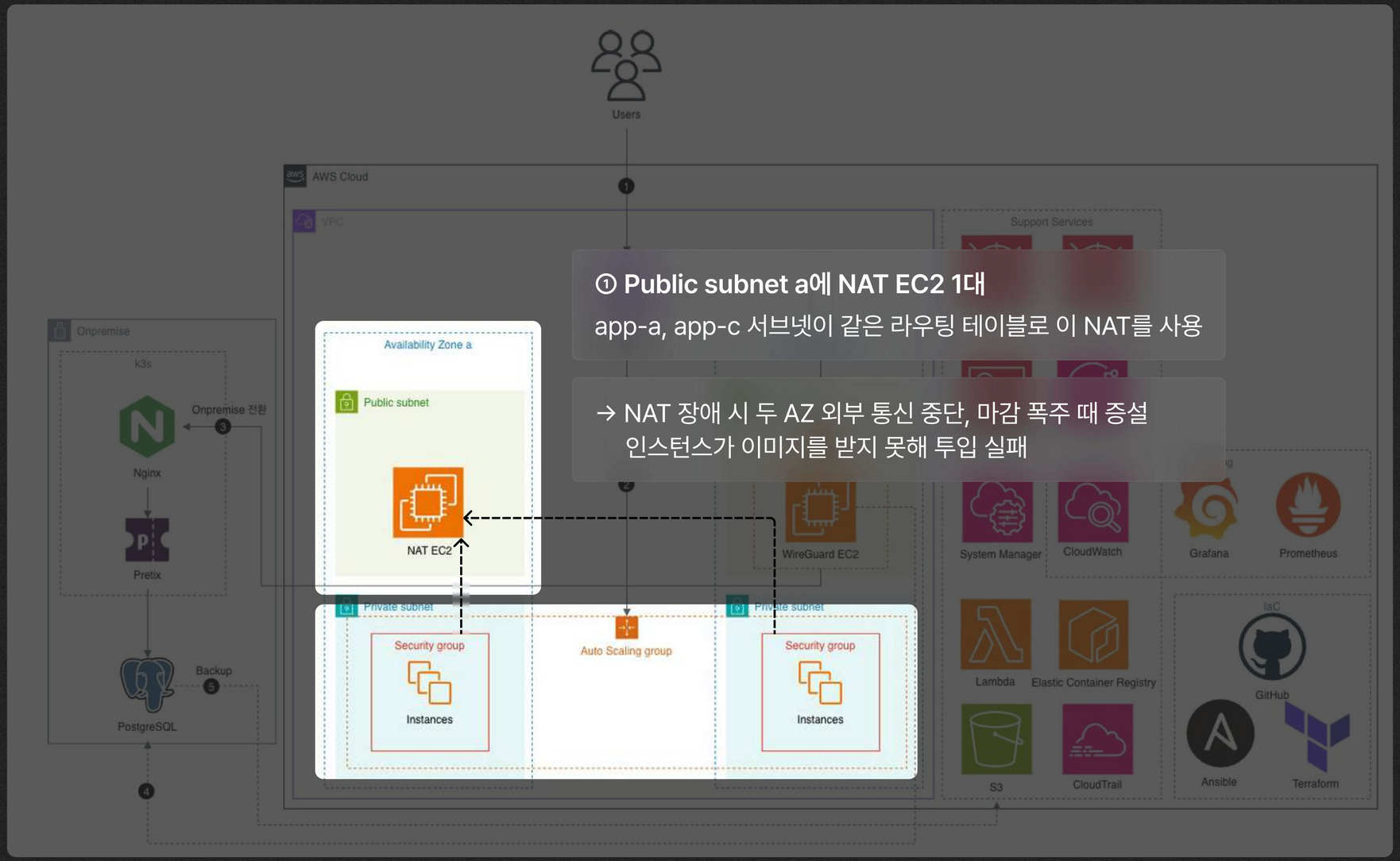
② 환경 재현 및 구현

고객사의 AWS 네트워크 구조 중 개선 대상 구간만 재현
일정상 전체 재현이 어려운 부분은 경량 구성으로 대체

1.3 현황 분석 AWS(ALB, ASG)와 온프레미스(Nginx, Pretix, PostgreSQL)를 WireGuard로 연결한 하이브리드 구성



1.3 현황 분석 두 AZ가 NAT 1대를 공유

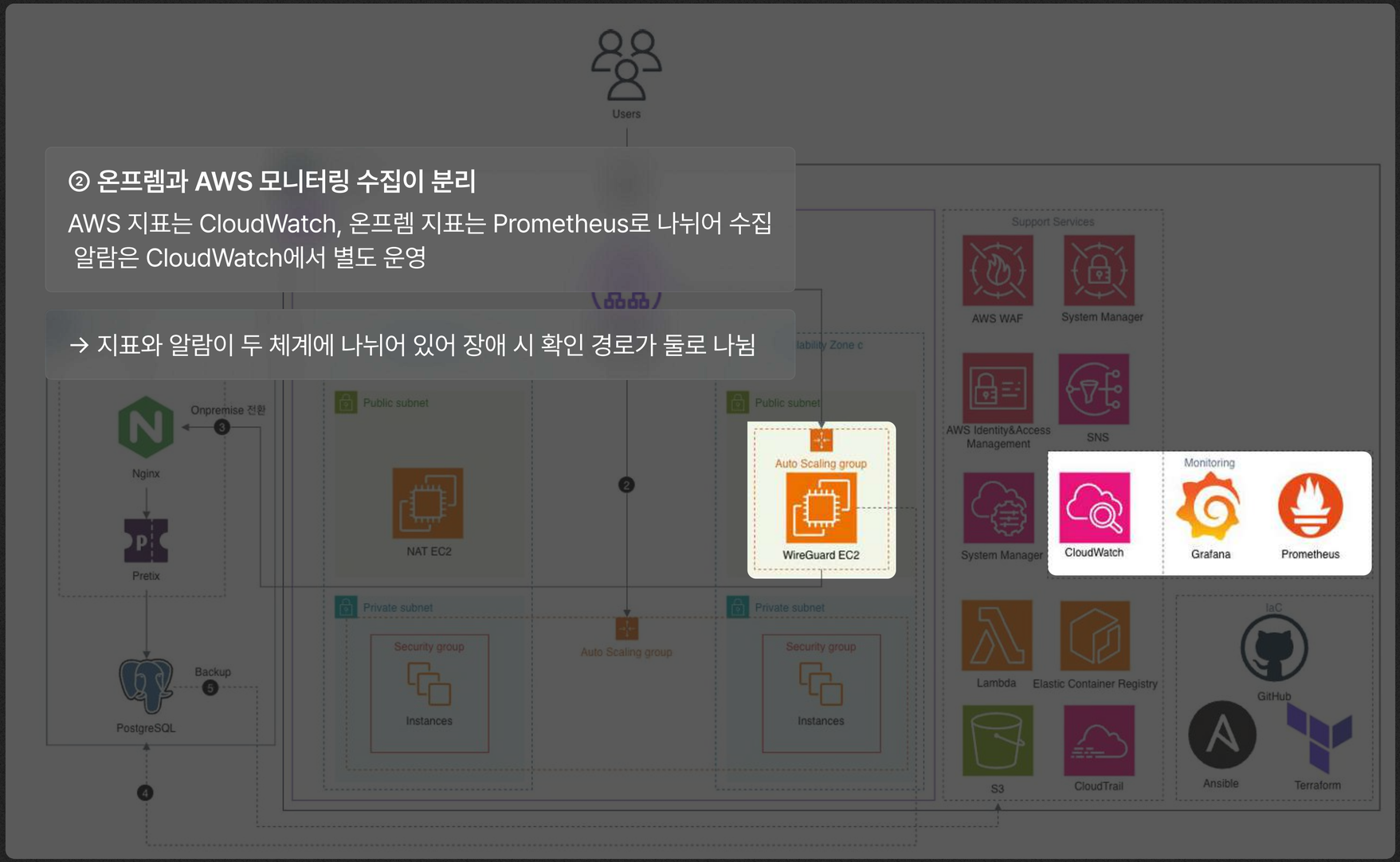


1.3 현황 분석 모니터링 수집 분리

② 온프렘과 AWS 모니터링 수집이 분리

AWS 지표는 CloudWatch, 온프렘 지표는 Prometheus로 나뉘어 수집
알람은 CloudWatch에서 별도 운영

→ 지표와 알람이 두 체계에 나뉘어 있어 장애 시 확인 경로가 둘로 나뉨



1.4 요구사항 정의

요구사항

6조의 주제인 트래픽 폭주 대응에 직접 영향을 주는 R1을 먼저, 운영 효율과 관련된 R2를 다음으로 진행

번호	요구사항	우선순위	근거
R1	NAT 1대에 장애가 나도 다른 AZ 앱 서버의 외부 통신이 유지되어야 하며, 장애 발생을 바로 알 수 있어야 한다	1	6조 추후 개선점 1
R2	AWS와 온프렘 지표를 하나의 수집 체계에서 확인할 수 있어야 한다	2	6조 추후 개선점 5

제약 조건

- 예산 월 $30, 현재 실사용 $49.52로 이미 초과 → 추가 비용을 최소화하는 구성
- 하이브리드 구조와 ALB 전환 방식 유지
- 3일 내 PoC 검증 → 개선 대상인 네트워크 구간만 재현, DB는 구현 범위에서 제외

2.1 요구사항별 솔루션

요구사항 01

NAT 단일 장애점 제거

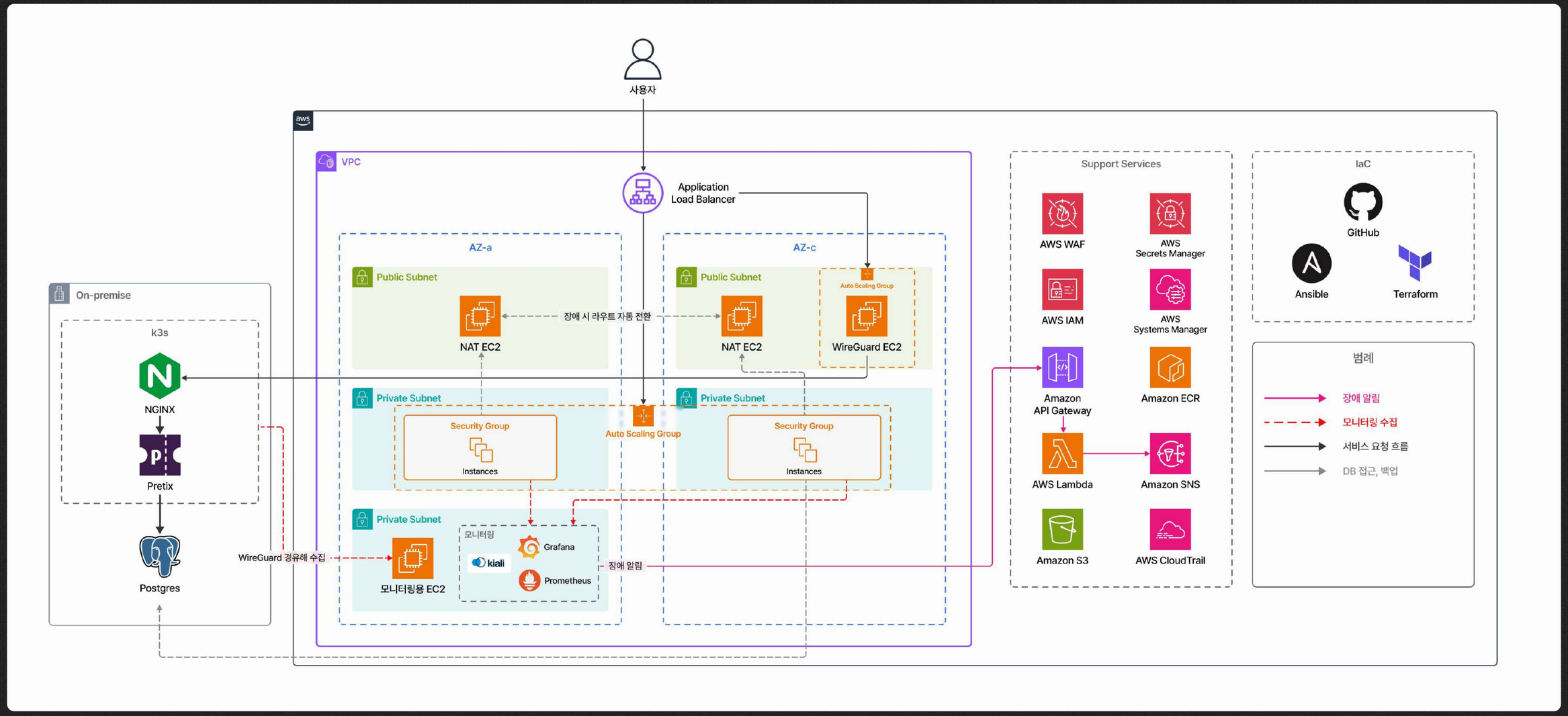
- AZ별로 NAT를 1대씩 두고 라우팅 테이블을 분리
- NAT 장애 시 라우트를 반대편 NAT로 자동 전환

요구사항 02

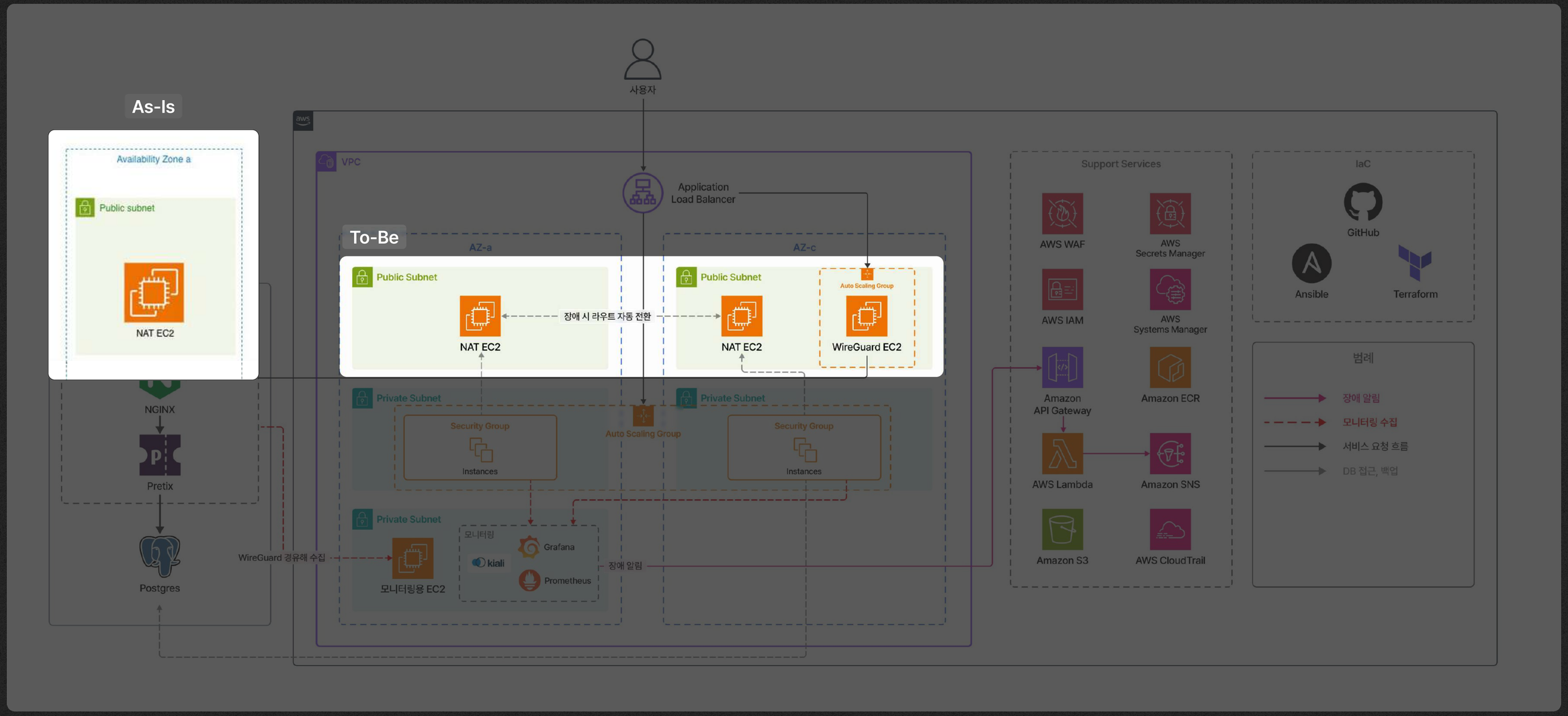
모니터링 통합

- AWS와 온프렘 지표를 Prometheus로 모아 Grafana에서 확인
- 알림 기준도 Grafana로 일원화

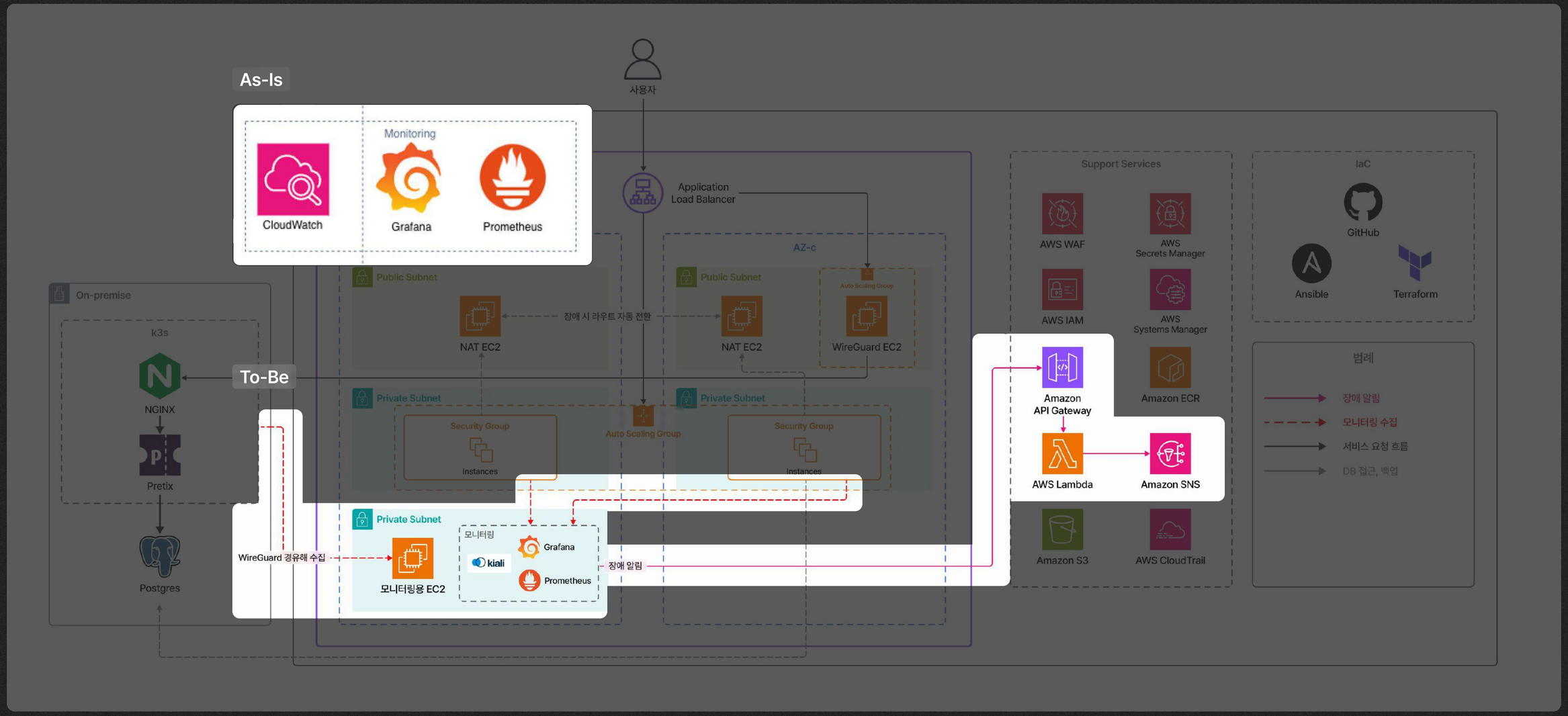
2.1 As-Is · To-Be NAT와 모니터링 구간만 변경 | 변경하지 않는 구성: ALB 가중치 전환, WireGuard 허브, 온프렘 k3s와 DB



2.1 As-Is · To-Be AZ별 NAT로 분리, 장애 시 반대편 NAT로 자동 전환



2.1 As-Is · To-Be 수집은 Prometheus, 알림은 Grafana로 일원화



2.2 기술 선택 근거

항목	검토한 대안	최종 선택	선택 이유
NAT 방식	AZ별 NAT Gateway	AZ별 NAT Instance	NAT Gateway 2개는 시간 요금만 월 약 $86로 예산 초과
NAT 장애 대응	Failover, Failback 모두 자동	Failover 자동, Failback 수동	Failover 자동화로 중단 시간 최소화, Failback은 경로 오류 위험이 있어 확인 후 수동으로
모니터링 지표 수집	CloudWatch 단일, 관리형 Grafana	Prometheus + Grafana	CloudWatch는 온프렘 서버마다 Agent 설치가 필요하고 커스텀 지표 수만큼 과금 관리형 Grafana를 써도 Prometheus와 Kiali를 돌릴 서버는 따로 필요 → 온프렘에서 쓰던 Prometheus로 AWS 지표까지 수집
장애 위치 파악	서비스 메시 없이 Prometheus 지표만	Istio + Kiali	앱 수정 없이 서비스 간 요청량, 에러율, 지연 수집. 사이드카만큼 노드 리소스 사용 증가
알림 경로	CloudWatch 알람 유지	Grafana → API Gateway → Lambda → SNS	알림 기준을 Grafana로 일원화, 기존 Lambda와 SNS 재사용

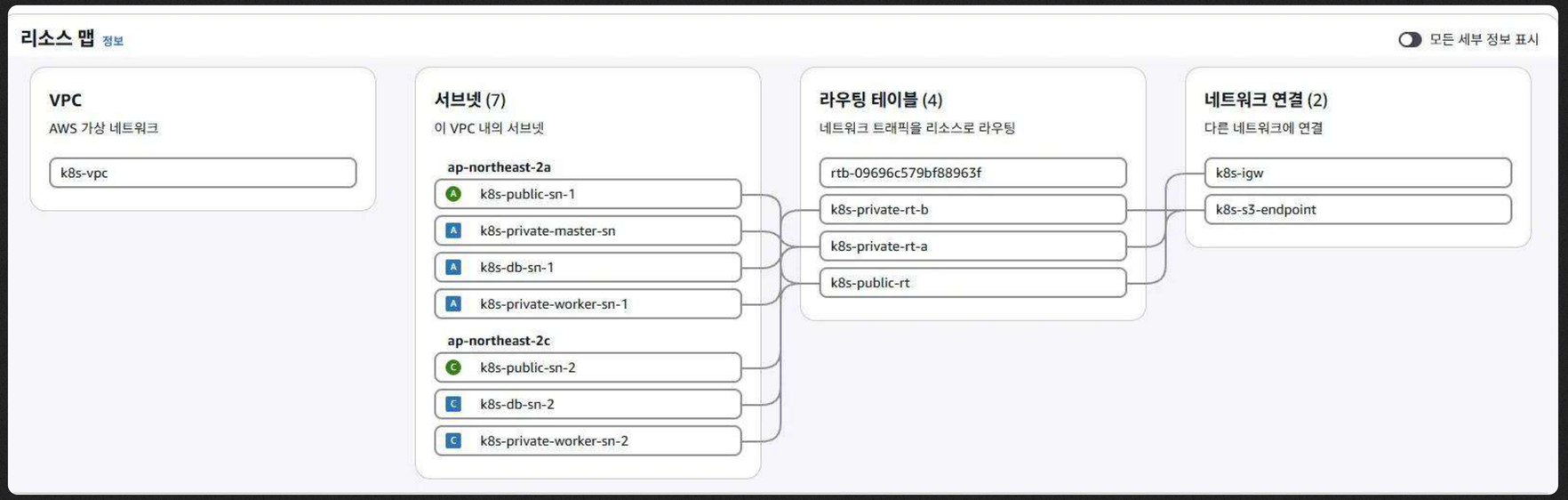
3.1 구현 결과 요약

① NAT Multi-AZ 구성과 자동 Failover 요구사항 01

AZ별 NAT Instance 1대씩 배치

Name	인스턴스 ID	인스턴스 상태	인스턴스 유형	상태 검사	가용 영역	퍼블릭 IPv4 DNS
k8s-nat-instance-A	i-09f209d9a1d8ea10a	실행 중	t3.nano	3/3 검사 통과	ap-northeast-2a	ec2-43-203-122-198.ap...
k8s-nat-instance-B	i-01bdba086c05201e0	실행 중	t3.nano	3/3 검사 통과	ap-northeast-2c	ec2-54-180-142-66.ap-...

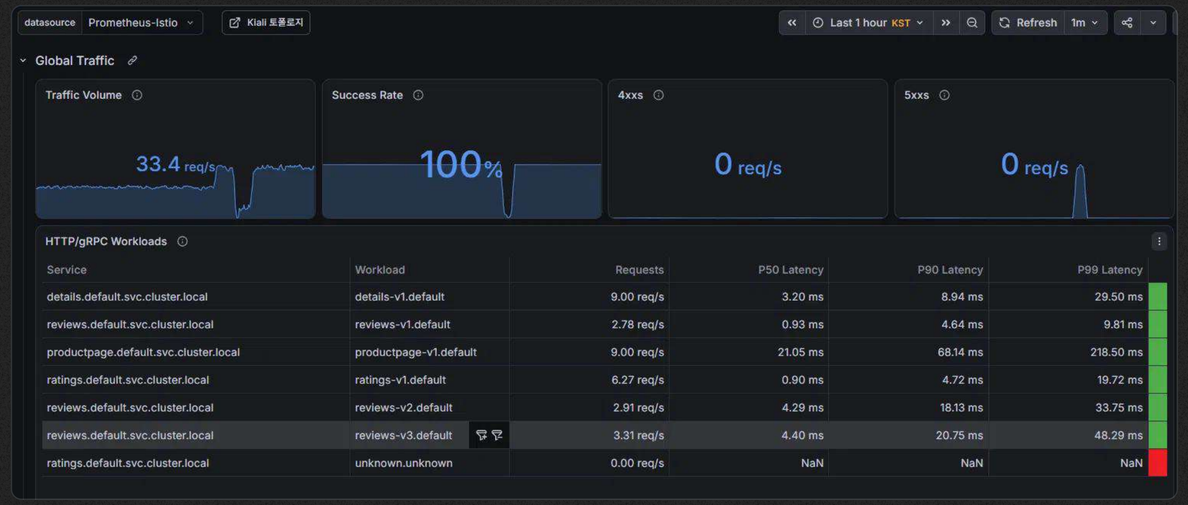
AZ별 프라이빗 라우팅 테이블 분리



3.1 구현 결과 요약

② 모니터링 통합 요구사항 02 | 서비스 지표와 서버 지표를 Prometheus로 수집해 Grafana 한 곳에서 확인

서비스 지표 · Istio Mesh Dashboard



서비스 간 요청량, 성공률, 에러(5xx), 응답 시간 확인

서버 지표 · Node Exporter Full

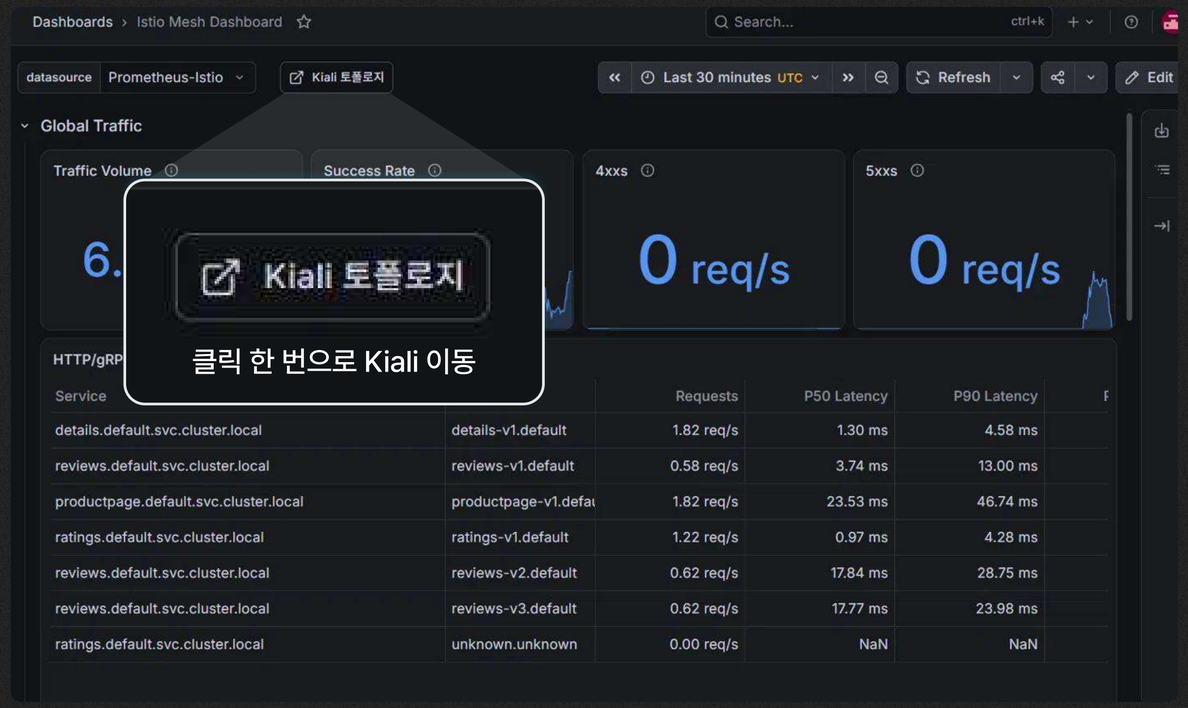


AWS 서버의 CPU, 메모리, 디스크, 네트워크 사용량 확인

3.1 구현 결과 요약

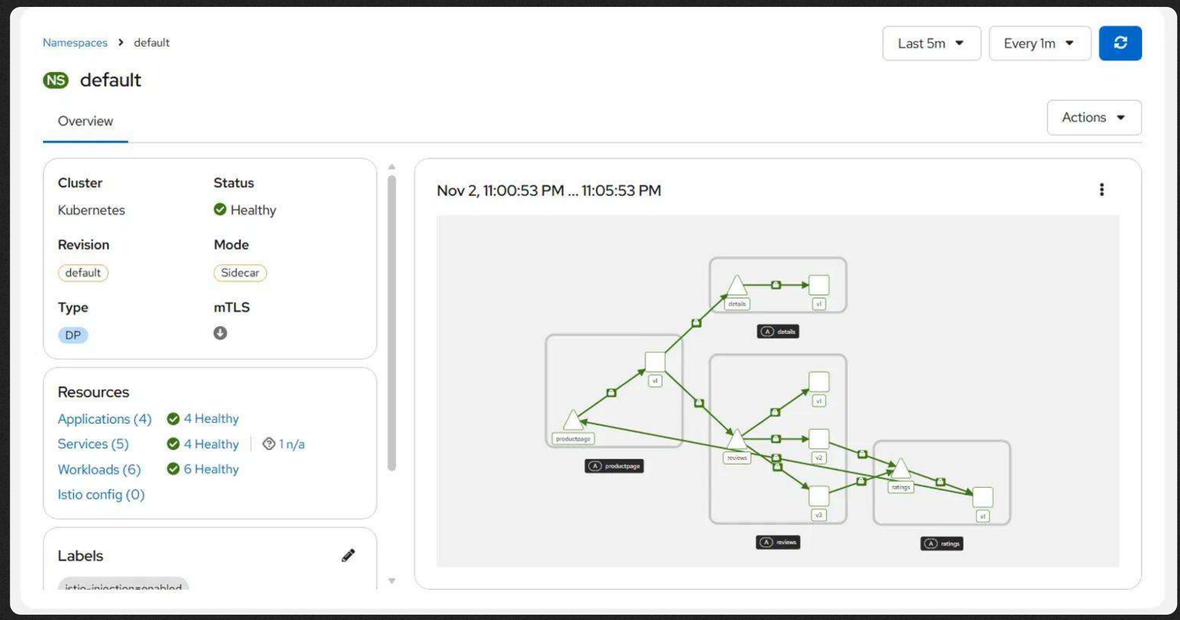
② 모니터링 통합 요구사항 02 | Grafana에서 이상을 확인하고, Kiali에서 문제가 생긴 서비스 위치를 확인

Grafana 대시보드



대시보드 상단 버튼으로 Kiali 토폴로지 화면에 바로 이동

Kiali 토폴로지



서비스 간 호출 관계와 상태를 색으로 표시 (초록 정상, 노랑 경고, 빨강 장애)

3.1 구현 결과 요약

② 모니터링 통합 요구사항 02 | 장애가 발생하면 Grafana가 감지해 이메일로 알림







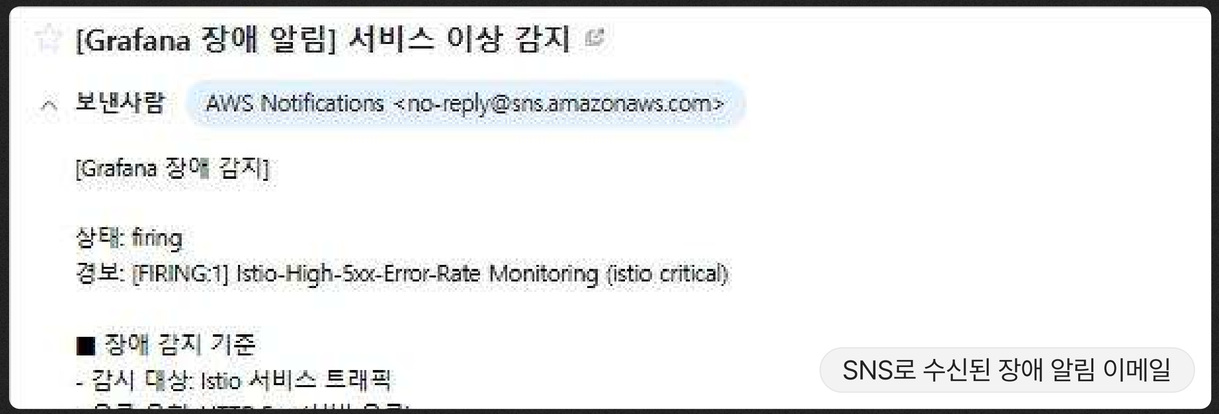
※ 초록 선: 서버 오류(5xx) 비율 / 빨간 선: 알림 기준 5%

검증 결과

① HTTP 500 에러 주입 후 서버 오류 비율 최고 약 15.5%까지 상승

② 기준 5%를 넘자 Grafana 알림이 Pending → Firing으로 전환

③ API Gateway → Lambda → SNS를 거쳐 이메일 수신

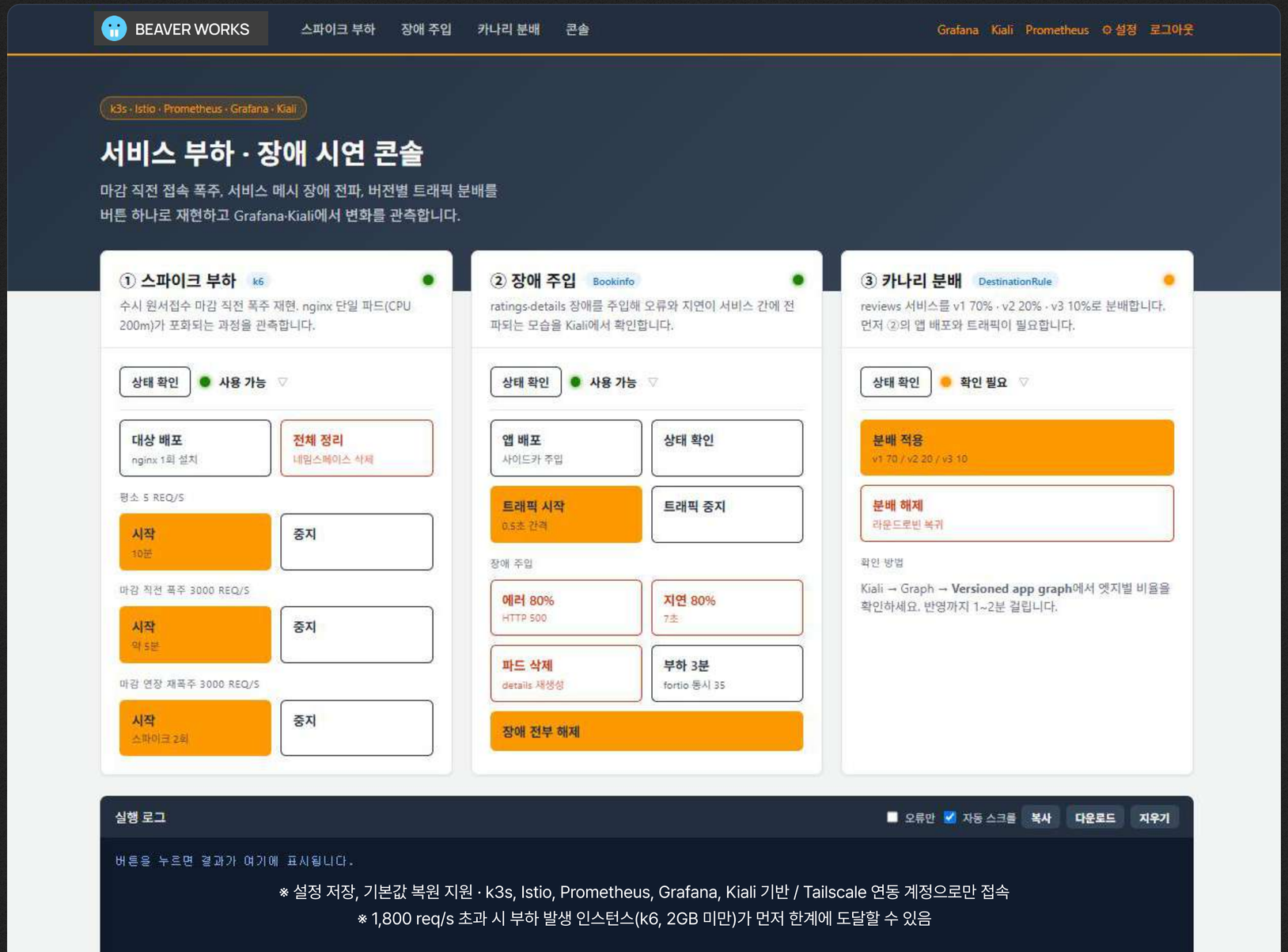


SNS로 수신된 장애 알림 이메일

3.2 시연 환경 세팅 접수 마감 트래픽과 장애 상황을 시연하기 위한 부하 테스트 콘솔

① 스파이크 부하

접수 마감 직전처럼 짧은 시간 동안 트래픽이 급증하는 상황을 재현



※ 설정 저장, 기본값 복원 지원 · k3s, Istio, Prometheus, Grafana, Kiali 기반 / Tailscale 연동 계정으로만 접속

※ 1,800 req/s 초과 시 부하 발생 인스턴스(k6, 2GB 미만)가 먼저 한계에 도달할 수 있음

② 장애 주입

특정 서비스에 에러와 지연을 발생시켜 장애 전파 확인

③ 카나리 분배

새 버전에 트래픽 일부만 보내 배포 안정성 확인

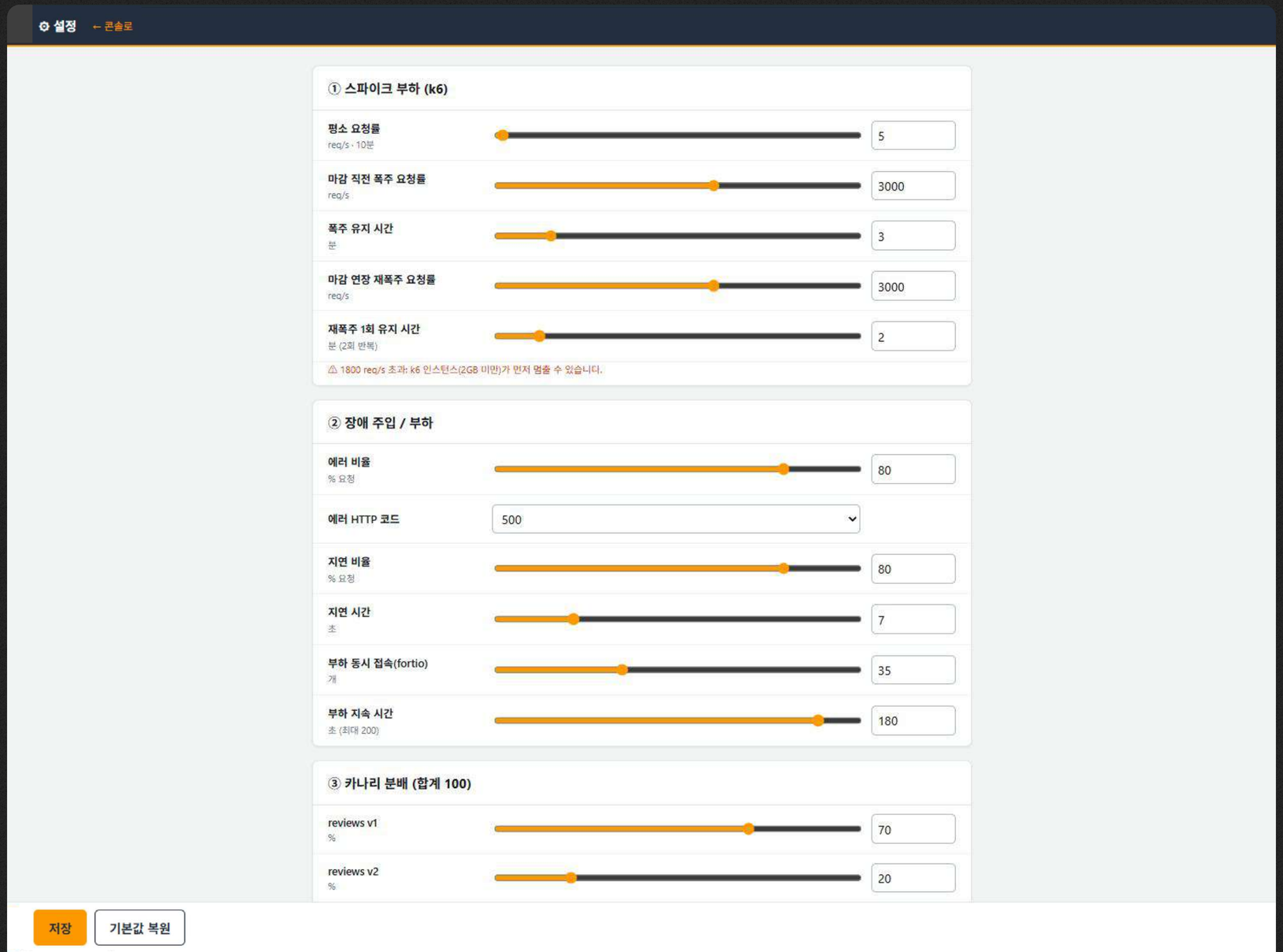
3.2 시연 환경 세팅 시나리오별 부하량과 장애 조건을 조정해 반복 테스트

※ 설정 저장, 기본값 복원 지원, k3s, Istio, Prometheus, Grafana, Kiali 기반

Tailscale 연동 계정으로만 접속

※ 1,800 req/s 초과 시 부하 발생

인스턴스(k6, 2GB 미만)가 먼저 한계에 도달할 수 있음



① 스파이크 부하 (k6)

요청률과 폭주 유지 시간 조정

기본 5 → 3,000 req/s, 3분

② 장애 주입 (Fortio)

에러, 지연 비율과 동시 접속 조정

기본 에러 80%, 지연 80%(7초), 35명

③ 카나리 분배

버전별 트래픽 비율 조정

기본 v1 70%, v2 20%, v3 10%

3.3 시연 영상

https://youtu.be/CXpMm-mIyaY

4.1 개선 효과 NAT 장애에도 서비스 유지, 모니터링은 한 체계로 통합

항목	기존 → 개선 후	고객에게 주는 효과
NAT 장애 대응	복구 전까지 두 AZ 외부 통신 중단 → 약 0.3초 내 반대편 NAT로 자동 전환	마감 폭주 중에도 증설 인스턴스 정상 투입
지표 수집	CloudWatch, Prometheus 2곳 → Prometheus 1곳	AWS와 온프렘 지표를 한 체계에서 수집
장애 위치 파악	요청 수, 송수신량 등 총량만 확인 → Kiali에서 문제 서비스 바로 확인	원인 서비스 추적 시간 단축
알림 경로	지표와 분리된 CloudWatch 알람 → 에러율 5% 초과 시 Grafana가 감지해 메일 자동 발송	지표와 같은 곳에서 알림 관리

4.2 비용 및 운영 방안

NAT Gateway 방식보다 적은 추가 비용으로 이중화와 통합 모니터링을 제공

월 비용

항목	월 비용
기존 실사용	$49.52
NAT Instance 1대 추가 (t3.nano)	+$4.75
모니터링 EC2 (t3.medium)	+$38
API Gateway, Lambda, SNS	≈ $0
개선 후 합계	**약 $92**

AZ 이중화 방식별 비용

- NAT Gateway 2대 — $ 86+
- NAT Instance 2대 — $ 9.5

운영 방안

도입 순서: NAT 이중화 → 모니터링 수집 통합 → 알림 전환

① NAT 장애 대응

Failover는 라우팅을 반대편 NAT로 자동 전환
Failback은 원래 NAT 정상 여부를 확인한 뒤 수동 전환

② 모니터링 서버 관리

모니터링 EC2가 멈추면 수집과 알림이 함께 멈춤
→ AMI로 백업해 두고 장애 시 같은 구성으로 재구축

③ 정기 점검

NAT Instance는 NAT Gateway와 달리 OS 패치 필요
NAT OS 패치, 모니터링 AMI 갱신을 주기적으로 수행

Q & A

BEAVER WORKS